

following line. Programmers will argue the merits of their preferred style until the end of time. Python neatly sidesteps this issue by not using braces at all, letting Pythonistas get back to more productive work. I've come to wish that all programming languages would adopt Python's approach to grouping blocks of code.

But some people still long for braces and want to add them to a future version of Python—despite how unpythonic they are. Python's `__future__` module backports features to earlier Python versions, and you'll find a hidden Easter egg if you try to import a braces feature into Python:

---

```
>>> from __future__ import braces
SyntaxError: not a chance
```

---

I wouldn't count on braces being added to Python any time soon.

## Commonly Misused Syntax

If Python isn't your first programming language, you might write your Python code with the same strategies you use to write code in other programming languages. Or perhaps you learned an unusual way of writing your Python code because you were unaware that there are more established best practices. This awkward code works, but you could save some time and effort by learning more standard approaches to writing pythonic code. This section explains common missteps programmers make and how you should write the code instead.

### ***Use `enumerate()` Instead of `range()`***

When looping over a list or other sequence, some programmers use the `range()` and `len()` functions to generate the index integers from 0 up to, but not including, the length of the sequence. It's common to use the variable name `i` (for *index*) in these `for` loops. For example, enter the following unpythonic example into the interactive shell:

---

```
>>> animals = ['cat', 'dog', 'moose']
>>> for i in range(len(animals)):
...     print(i, animals[i])
```

---